



空间数据库原理与应用

第三讲：关系数据库标准语言SQL（一）

任课教师：李晓明

建筑与城市规划学院 城市空间信息工程系

其中部分图片来自互联网和同行专家

目录

CONTENTS

01 SQL概述

02 数据定义

03 简单查询

04 数据更新

目录

CONTENTS

01

SQL概述

1.1 概述

1.2 特点

1.3 SQL数据库的体系结构

1.1 概述

SQL概述

- **SQL(Structured Query Language)结构化查询语言，1974年Boyce和Chamberlin提出，首先在IBM 公司的关系数据库系统System R上实现。**
- **特点：功能丰富、使用方便、灵活、语言简洁易学、应用系统范围广、统一标准。**
- **1986年，ANSI数据库委员会X3H2批准了SQL作为数据库语言的美国标准，ISO随后也提出了同样的决定。**

1.1 概述

SQL标准

- 有关组织

美国国家标准学会ANSI(American Natural Standard Institute)

国际标准化组织ISO(International Organization for Standardization)

- 有关标准

SQL-86: “数据库语言SQL”

SQL-89: “具有完整性增强的数据库语言SQL”，增加了对完整性约束的支持

SQL-92: “数据库语言SQL”，是SQL-89的超集，增加了许多新特性，如新的数据类型，更丰富的数据操作，更强的完整性、安全性支持等。

SQL-99

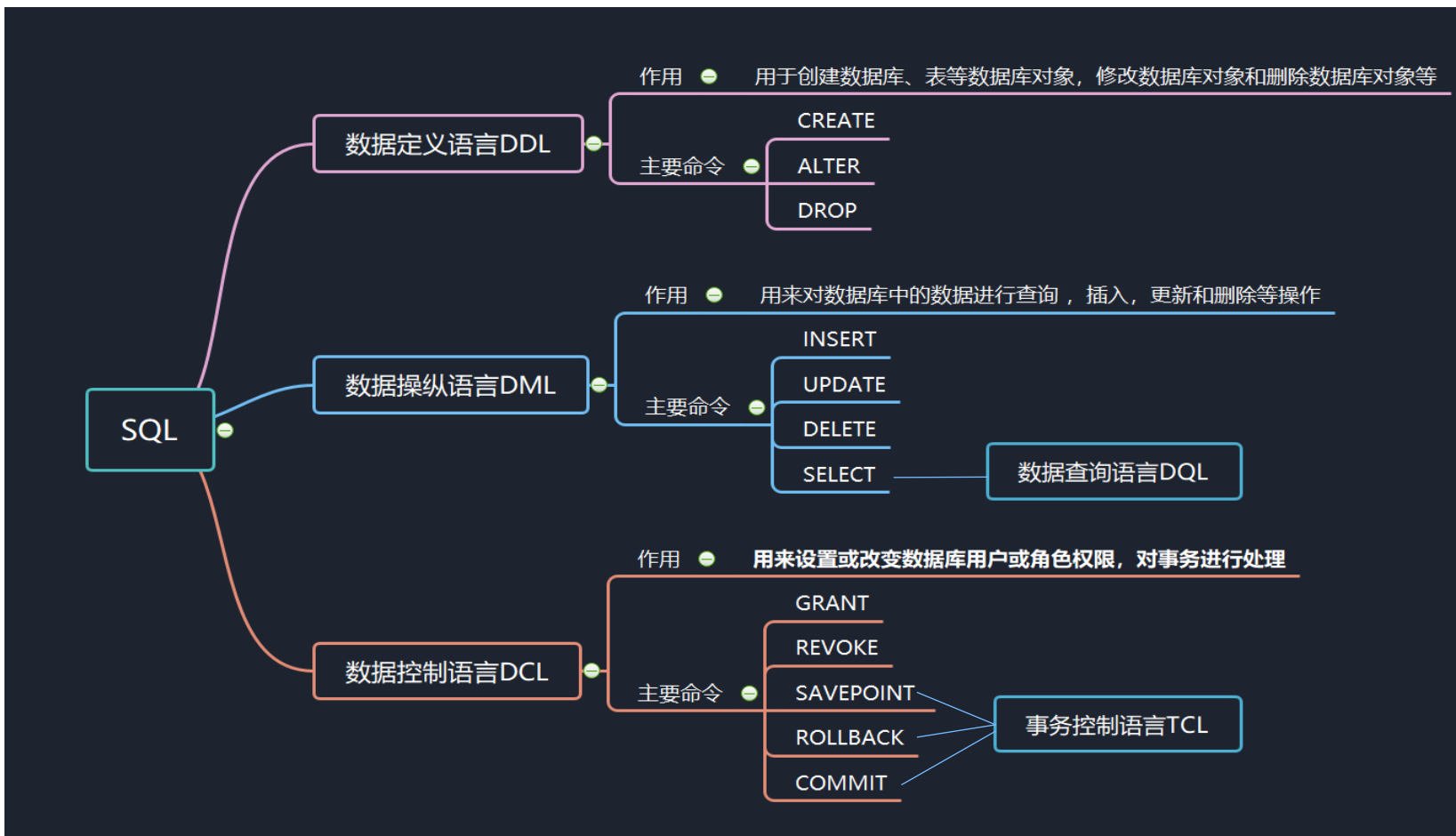
SQL-2003

- 大部分DBMS产品都支持SQL，成为操作数据库的标准语言

- 不同DBMS，支持程度有所不同（因为不同的DBMS，在SQL语句的设计和语句用法上存在差异，就好比不同地方的人说不同的方言）

1.1 概述

SQL分类



交互式SQL→嵌入式SQL→动态SQL

目录

CONTENTS

2

建立数据库

2.1 数据库定义和删除

2.2 基本表的定义和删除

2.3 向表中追加元组的值

2.1 数据库的定义和删除

数据库的创建

建立数据库

➤ 包括两件事：定义数据库和表（使用DDL），向表中追加元组（使用DML）

DDL: Data Definition Language

- ❑ 创建数据库(DB)—Create Database
- ❑ 创建DB中的Table(定义关系模式)---Create Table
- ❑ 定义Table及其各个属性的约束条件(定义完整性约束)
- ❑ 定义View (定义外模式及E-C映像)
- ❑ 定义Index、Tablespace...等(定义物理存储参数)
- ❑ 上述各种定义的撤消与修正
- DDL通常由DBA来使用，也有经DBA授权后由应用程序员来使用

2.1 模式定义和删除

数据库的创建

创建数据库SQL语句:

create database 数据库名;

示例: 创建课程学习数据库SCT

create database SCT;

删除数据库SQL语句:

DROP DATABASE [IF EXISTS] 数据库名;

示例: 删除课程学习数据库SCT

drop database SCT;

2.2 基本表的定义和删除

基本表的定义

创建Table

- **create table** 简单语法形式:

Create table 表名(列名 数据类型 [**Primary key | Unique**] [**Not null**]
[, 列名 数据类型 [**Not null**] , ...]);

- “**[]**”表示其括起的内容可以省略, “**|**”表示其隔开的两项可取其一
- **Primary key**: 主键约束。每个表只能创建一个主键约束。
- **Unique**: 唯一性约束(即候选键)。可以有多个唯一性约束。
- **Not null**: 非空约束。是指该列允许不允许有空值出现, 如选择了**Not null**表明该列不允许有空值出现。
- 语法中的数据类型在**SQL**标准中有定义

2.2 基本表的定义和删除

基本表的定义

■ SQL-92标准中定义的数据类型

char (n) :固定长度的字符串

varchar (n) :可变长字符串

int :整数// 有时不同系统也写作integer

numeric (p, q) :固定精度数字, 小数点左边p位, 右边p-q位

real :浮点精度数字//有时不同系统也写作float(n), 小数点后保留n位

date :日期(如2003-09-12)

time : 时间(如23:15:003)

...

现行商用DBMS的数据类型有时和上面有些差异, 请注意;
和高级语言的数据类型, 总体上是一致的, 但也有些差异。

2.2 基本表的定义和删除

基本表的定义

示例：定义学生表 Student

S# char(8), **Sname** char(10), **Ssex** char(2), **Sage** integer, **D#** char(2), **Sclass** char(6)

Create Table Student (**S#** char(8) not null , **Sname** char(10),
Ssex char(2), **Sage** integer, **D#** char(2), **Sclass** char(6));

示例：定义课程表 Course

C# char(3) , **Cname** char(12), **Chours** integer, **Credit** float(1), **T#** char(3)

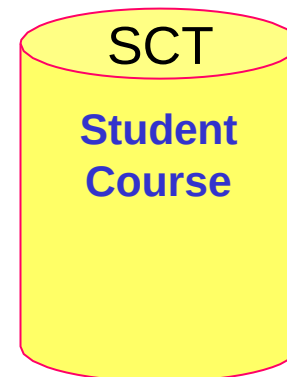
Create Table Course (**C#** char(3) , **Cname** char(12), **Chours** integer, **Credit** float(1), **T#** char(3));

Student

S#	Sname	Ssex	Sage	D#	Sclass
98030101	张三	男	20	03	980301
98030102	张四	女	20	03	980301
98030103	张五	男	19	03	980301
98040201	王三	男	20	04	980402
98040202	王四	男	21	04	980402
98040203	王五	女	19	04	980402

Course

C#	Cname	Chours	Credit	T#
001	数据库	40	6	001
003	数据结构	40	6	003
004	编译原理	40	6	001
005	C 语言	30	4.5	003
002	高等数学	80	12	004



2.2 基本表的定义和删除

删除基本表

撤消基本表

drop table 表名

示例：撤消学生表Student

```
Drop Table Student;
```

示例：撤消教师表Teacher

```
Drop Table Teacher;
```

➤注意，SQL-delete语句只是删除表中的元组, 而撤消基本表drop table的操作是撤消包含表格式、表中所有元组、由该表导出的视图等相关的所有内容，所以使用要特别注意。

2.2 基本表的定义和删除

修改基本表

alter table tablename

[**add** {colname datatype, ...}]

增加新列

[**drop** {完整性约束名}]

删除完整性约束

[**modify** {colname datatype, ...}]

修改列定义

示例：在学生表Student(S#,Sname,Ssex,Sage,D#,Sclass)基础上增加二列Saddr, PID, 分别是40长度和18长度的字符串

```
Alter Table Student Add Saddr char[40], PID char[18];
```

示例：将上列表中Sname列的数据类型, 增加到12个字符长度

```
Alter Table Student Modify Sname char(12);
```

示例：删除学生姓名必须取唯一值的约束

```
Alter Table Student Drop Unique( Sname );
```

2.3 向表中追加元组的值

建立数据库

包括两件事：定义数据库和表（使用DDL），向表中追加元组（使用DML）
DML: Data Manipulation Language

- ❑ 向**Table**中追加新的元组: **Insert**
- ❑ 修改**Table**中某些元组中的某些属性的值: **Update**
- ❑ 删除**Table**中的某些元组: **Delete**
- ❑ 对**Table**中的数据进行各种条件的检索: **Select**
- **DML通常由用户或应用程序员使用，访问经授权的数据库**

2.3 向表中追加元组的值

向表中追加元组

➤ **insert into** 简单语法形式:

insert into 表名[(列名[, 列名]...]
values (值[, 值] , ...);

- **values**后面值的排列, 须与**into**子句后面的列名排列一致
- 若表名后的所有列名省略, 则**values**后的值的排列, 须与该表存储中的列名排列一致

示例: 追加学生表中的元组

Insert Into Student

Values ('98030101', '张三', '男', 20, '03', '980301');

Insert Into Student (S#, Sname, Ssex, Sage, D#, Sclass)

Values ('98030102', '张四', '女', 20, '03', '980301');

Student					
S#	Sname	Ssex	Sage	D#	Sclass
98030101	张三	男	20	03	980301
98030102	张四	女	20	03	980301
98030103	张三	男	19	03	980301
98040201	王三	男	20	04	980402
98040202	王四	男	21	04	980402
98040203	王五	女	19	04	980402

示例: 追加课程表中的元组

*/*所有列名省略, 须与定义或存储的列名顺序一致*

Insert Into Course

Values ('001', '数据库', 40, 6, '001');

*/*如列名未省略, 须与语句中列名的顺序一致*

Insert Into Course(Cname, C#, Credit, Chours, T#)

Values ('数据库', '001', 6, 40, '001');

Course				
C#	Cname	Chours	Credit	T#
001	数据库	40	6	001
003	数据结构	40	6	003
004	编译原理	40	6	001
005	C 语言	30	4.5	003
002	高等数学	80	12	004

目录

CONTENTS

3

简单查询

3.1 单表查询

3.2 连接查询

SELECT语句的基本语法

SQL提供了结构形式一致但功能多样化的检索语句Select

➤ **Select** 的简单语法形式:

Select 列名 [[, 列名] ...]

From 表名

[**Where** 检索条件] ;

➤ 语义: 从表名所给出的表中, 查询出满足检索条件的元组, 并按给定的列名及顺序进行投影显示。

➤ 相当于: $\Pi_{\text{列名}, \dots, \text{列名}} (\sigma_{\text{检索条件}} (\text{表名}))$

➤ **Select**语句中的**select ...**, **from...**, **where...**, 等被称为子句, 在以上基本形式基础上会增加许多构成要素, 也会增加许多新的子句, 满足不同的需求。



SELECT语句的基本语法

示例：检索学生表中所有学生的信息

```
Select  S#, Sname, Ssex, Sage, Sclass, D#  
From Student ;
```

```
Select  * From Student ;
```

//如投影所有列，则可以用*来简写

示例：检索学生表中所有学生的姓名及年龄

```
Select  Sname, Sage  
From Student ;
```

//投影出某些列

示例：检索学生表中所有年龄小于等于19岁的学生的年龄及姓名

```
Select  Sage, Sname  
From Student  
Where  Sage <= 19;
```

//投影的列可以重新排定顺序

Student

S#	Sname	Ssex	Sage	D#	Sclass
98030101	张三	男	20	03	980301
98030102	张四	女	20	03	980301
98030103	张五	男	19	03	980301
98040201	王三	男	20	04	980402
98040202	王四	男	21	04	980402
98040203	王五	女	19	04	980402

SELECT-FROM-WHERE句型

检索条件的书写

➤与选择运算 $\sigma_{con}(R)$ 的条件con书写一样，只是其逻辑运算符用 **and** , **or**, **not** 来表示，同时也要注意运算符的优先次序及括弧的使用。书写要点是注意对自然语言检索条件的正确理解。

示例：检索教师表中所有工资少于1500元或者工资大于2000元 并且是03系的教师姓名？

Select Tname From Teacher

Where Salary < 1500 or Salary > 2000 and D# = '03';

Select Tname

From Teacher

Where (Salary < 1500 or Salary > 2000) and D# = '03';

Teacher

T#	Tname	D#	Salary
001	赵三	01	1200.00
002	赵四	03	1400.00
003	赵五	03	1000.00
004	赵六	04	1100.00

3.1 单表查询

检索条件的书写

常用的查询条件

查询条件	谓词
比较	=, >, <, >=, <=, !=, !<, NOT+上述比较运算
确定范围	BETWEEN AND
确定集合	IN, NOT IN
字符匹配	LIKE, NOT LIKE
空值	IS NULL, IS NOT NULL
多重条件	AND, OR

说明：查询满足指定条件的元组可以通过 WHERE 子句实现。

3.1 单表查询

结果唯一性问题

重复元组的处理

- 语法约束：缺省为保留重复元组，也可用关键字**all**显式指明。若要去掉重复元组，可用关键字**distinct**或**unique**指明。

示例：在选课表中，检索成绩大于80分的所有学号

Select S#

From SC

Where Score > 80 ;

//有重复元组出现，比如一个同学两门以上课程大于80

Select DISTINCT S#

From SC

Where Score > 80;

//重复元组被DISTINCT过滤掉，只保留一份

SC		
S#	C#	Score
98030101	001	92
98030101	002	85
98030101	003	88
98040202	002	90
98040202	003	80
98040202	001	55
98040203	003	56
98030102	001	54
98030102	002	85
98030102	003	48



3.1 单表查询

结果排序问题

DBMS可以对检索结果进行排序，可以升序排列，也可以降序排列。

命令 **order by 列名 [asc | desc]**

ORDER BY 子句表示结果要排序,它必须在所有其它子句之后作为最后一个子句出现。

示例：按学号由小到大的顺序显示出所有学生的学号及姓名

Select S#, Sname From Student
Order By S# ASC ;

示例：检索002号课大于80分的所有同学学号并按成绩由高到低顺序显示

Select S# From SC Where C# = '002' and Score > 80
Order By Score DESC ;

Student					
S#	Sname	Ssex	Sage	D#	Sclass
98030101	张三	男	20	03	980301
98030102	张四	女	20	03	980301
98030103	张五	男	19	03	980301
98040201	王三	男	20	04	980402
98040202	王四	男	21	04	980402
98040203	王五	女	19	04	980402

SC		
S#	C#	Score
98030101	001	92
98030101	002	85
98030101	003	88
98040202	002	90
98040202	003	80
98040202	001	55
98040203	003	56
98030102	001	54
98030102	002	85
98030102	003	48

3.1 单表查询

模糊查询问题

模糊查询问题

比如检索姓张的学生，检索张某某；这类查询问题，**Select**语句是通过在检索条件中引入运算符**like**来表示的

➤ 含有**like**运算符的表达式

列名 [**not**] **like** “字符串”

➤ 找出匹配给定字符串的字符串。其中给定字符串中可以出现%, _等匹配符.

➤ 匹配规则:

- ☐ “**%**” 匹配零个或多个字符
- ☐ “**_**” 匹配任意单个字符
- ☐ “****” 转义字符，用于去掉一些特殊字符的特定含义，使其被作为普通字符看待，如用 “**\%**”去匹配字符%，用**_** 去匹配字符_



3.1 单表查询

模糊查询问题

示例：检索所有姓张的学生学号及姓名

```
Select S#, Sname From Student  
Where Sname Like '张%';
```

示例：检索名字为张某某的所有同学姓名

```
Select Sname From Student  
Where Sname Like '张__';
```

示例：检索名字不姓张的所有同学姓名

```
Select Sname From Student  
Where Sname Not Like '张%';
```

Student

S#	Sname	Ssex	Sage	D#	Sclass
98030101	张三	男	20	03	980301
98030102	张四	女	20	03	980301
98030103	张五	男	19	03	980301
98040201	王三	男	20	04	980402
98040202	王四	男	21	04	980402
98040203	王五	女	19	04	980402

3.2 连接查询

多表联合查询

多表联合检索可以通过连接运算来完成，而连接运算又可以通过广义笛卡尔积后再进行选择运算来实现。

➤ **Select** 的多表联合检索语句

Select 列名 [[, 列名] ...]

From 表名1, 表名2, ...

Where 检索条件 ;

➤ 相当于 $\Pi_{\text{列名}, \dots, \text{列名}} (\sigma_{\text{检索条件}} (\text{表名1} \times \text{表名2} \times \dots))$

➤ 检索条件中要包含连接条件，通过不同的连接条件可以实现等值连接、不等值连接及各种 θ -连接



3.2 连接查询

等值连接

示例：按 “001”号课成绩由高到低顺序显示所有学生的姓名(二表连接)

```
Select Sname From Student, SC
Where Student.S# = SC.S# and SC.C# = '001'
Order By Score DESC;
```

➤多表连接时，如两个表的属性名相同，则需采用表名.属性名方式来限定该属性是属于哪一个表

示例：按 ‘数据库’ 课成绩由高到低顺序显示所有同学姓名(三表连接)

```
Select Sname From Student, SC, Course
Where Student.S# = SC.S# and SC.C# = Course.C#
and Cname = '数据库'
Order By Score DESC;
```

Student					
S#	Sname	Ssex	Sage	D#	Sclass
98030101	张三	男	20	03	980301
98030102	张四	女	20	03	980301
98030103	张五	男	19	03	980301
98040201	王三	男	20	04	980402
98040202	王四	男	21	04	980402
98040203	王五	女	19	04	980402

Course				
C#	Cname	Chours	Credit	T#
001	数据库	40	6	001
003	数据结构	40	6	003
004	编译原理	40	6	001
005	C 语言	30	4.5	003
002	高等数学	80	12	004

SC		
S#	C#	Score
98030101	001	92
98030101	002	85
98030101	003	88
98040202	002	90
98040202	003	80
98040202	001	55
98040203	003	56
98030102	001	54
98030102	002	85
98030102	003	48

3.2 连接查询

重名之处理

连接运算涉及到重名的问题，如两个表中的属性重名，连接的两个表重名(同一表的连接)等，因此需要使用**别名**以便区分

➤ **select**中采用别名的方式

Select 列名**as** 列别名[[, 列名**as** 列别名] ...]

From 表名**1 as** 表别名**1**, 表名**2 as** 表别名**2**, ...

Where 检索条件;

➤上述定义中的**as** 可以省略

➤当定义了别名后，在检索条件中可以使用别名来限定属性

示例：求年龄有差异的任意两位同学的姓名

Select S1.Sname **as** Stud1, S2.Sname **as** Stud2

From Student S1, Student S2

Where S1.Sage > S2.Sage ;

有时表名很长时，为书写条件简便，也定义表别名，以简化书写

3.2 连接查询

多表联合查询训练

示例：求既学过 “001”号课又学过 “002”号课的所有学生的学号

```
Select S1.S# From SC S1, SC S2
Where S1.S# = S2.S# and S1.C#='001'
and S2.C#='002' ;
```

示例：求 “001”号课成绩比 “002”号课成绩高的所有学生的学号

```
Select S1.S# From SC S1, SC S2
Where S1.S# = S2.S# and S1.C#='001'
and S2.C#='002' and S1.Score > S2.Score;
```

SC S1			SC S2		
S#	C#	Score	S#	C#	Score
98030101	001	92.0	98030101	001	92.0
98030101	002	85.0	98030101	002	85.0
98030101	003	88.0	98030101	003	88.0
98040202	002	90.5	98040202	002	90.5
98040202	003	80.0	98040202	003	80.0
98040202	001	55.0	98040202	001	55.0
98050104	003	56.0	98050104	003	56.0
98030102	001	54.0	98030102	001	54.0
98030102	002	85.0	98030102	002	85.0
98030102	003	48.0	98030102	003	48.0

目录

CONTENTS

4

数据更新

4.1 数据插入

4.2 数据修改

4.3 数据删除

数据更新操作

- 元组新增Insert: 新增一个或一些元组到数据库的Table中
 - 元组更新Update: 对某些元组中的某些属性值进行重新设定
 - 元组删除Delete: 删除某些元组
-
- SQL-DML既能单一记录操作, 也能对记录集合进行批更新操作
 - SQL-DML之更新操作需要利用前面介绍的子查询(Subquery)的概念, 以便处理“一些”、“某些”等。

4.1 数据插入

插入单个元组

➤ 元组新增Insert命令有两种形式

➤ 单一元组新增命令形式：插入一条指定元组值的元组

```
insert into 表名 [(列名[, 列名]...)]  
values (值[, 值]...);
```

➤ 批数据新增命令形式：插入子查询结果中的若干条元组。待插入的元组由子查询给出。

```
insert into 表名 [(列名[, 列名]...)]  
子查询;
```


4.1 数据插入

插入单个元组

示例：单一元组新增

Insert Into Teacher (T#, Tname, D#, Salary)
Values (“005”, “阮小七”, “03”, “1250”);

Insert Into Teacher
Values (“006”, “李小虎”, “03”, “950”);

Teacher

T#	Tname	D#	Salary
001	赵三	01	1200.00
002	赵四	03	1400.00
003	赵五	03	1000.00
004	赵六	04	1100.00

4.1 数据插入

批元组新增

➤ 示例:

新建立Table: **St(S#, Sname)**, 将检索到的满足条件的同学新增到该表中

```
Insert Into St (S#, Sname)
```

```
  Select S#, Sname From Student Where Sname like '%伟 '  
;
```

```
Insert Into St (S#, Sname)
```

```
  Select S#, Sname From Student Order By Sname;
```

Student

S#	Sname	Ssex	Sage	D#	Sclass
98030101	张三	男	20	03	980301
98030102	张四	女	20	03	980301
98030103	张五	男	19	03	980301
98040201	王三	男	20	04	980402
98040202	王四	男	21	04	980402
98040203	王五	女	19	04	980402

➤注意：当新增元组时，**DBMS**会检查用户定义的完整性约束条件等，如不符合完整性约束条件，则将不会执行新增动作



4.1 数据插入

插入子查询结果

示例：新建Table: **SCt(S#, C#, Score)**, 将检索到的成绩及格同学的记录新增到该表中

```
Insert Into SCt (S#, C#, Score)
Select S#, C#, Score From SC
Where Score>=60 ;
```

示例：追加成绩优秀同学（分数>=90）的记录

```
Insert Into SCt (S#, C#, Score)
Select S#, C#, Score From SC
Where Score>=90 ;
```

SC		
S#	C#	Score
98030101	001	92
98030101	002	85
98030101	003	88
98040202	002	90
98040202	003	80
98040202	001	55
98040203	003	56
98030102	001	54
98030102	002	85
98030102	003	48



4.1 数据插入

插入子查询结果

还可以有更复杂的“查询后插入到新表中”的语句，例如可以将中间结果存储成表---这很有用！

示例：新建Table: **St(S#, Sname, avgScore)**, 将检索到的同学的平均成绩新增到该表中

```
Insert Into St (S#, Sname, avgScore)
Select S#, Sname, Avg(Score) From Student, SC
Where Student.S# = SC.S#
Group by Student.S# ;
```

Student

S#	Sname	Ssex	Sage	D#	Sclass
98030101	张三	男	20	03	980301
98030102	张四	女	20	03	980301
98030103	张五	男	19	03	980301
98040201	王三	男	20	04	980402
98040202	王四	男	21	04	980402
98040203	王五	女	19	04	980402

SC

S#	C#	Score
98030101	001	92
98030101	002	85
98030101	003	88
98040202	002	90
98040202	003	80
98040202	001	55
98040203	003	56
98030102	001	54
98030102	002	85
98030102	003	48

4.2 数据删除

数据删除

- 元组删除Delete命令：删除满足指定条件的元组
Delete From 表名 [**Where** 条件表达式] ;
- 如果Where条件省略，则删除所有的元组。

示例：删除SC表中所有元组

```
Delete From SC ;
```

示例：删除98030101号同学所选的所有课程

```
Delete From SC Where S# = '98030101' ;
```

示例：删除自动控制系的所有同学

```
Delete From Student Where D# in
```

```
( Select D# From Dept Where Dname = '自动控制' );
```

---此是一简单的嵌套子查询，后面会有更详细解释。

SC		
S#	C#	Score
98030101	001	92
98030101	002	85
98030101	003	88
98040202	002	90
98040202	003	80
98040202	001	55
98040203	003	56
98030102	001	54
98030102	002	85
98030102	003	48

4.3 数据修改

数据修改

➤ 元组更新Update命令：用指定要求的值更新指定表中满足指定条件的 元组的指定列的值

Update 表名

Set 列名 = 表达式 | (子查询)

[[, 列名 = 表达式 | (子查询)] ...] [**Where**
条件表达式];

➤ 如果Where条件省略，则更新所有的元组。

示例：将所有教师工资上调5%

Update Teacher

Set Salary = Salary * 1.05 ;

Teacher

T#	Tname	D#	Salary
001	赵三	01	1200.00
002	赵四	03	1400.00
003	赵五	03	1000.00
004	赵六	04	1100.00

示例：将所有计算机系的教师工资上调10%

Update Teacher

Set Salary = Salary * 1.1 Where

D# in

(Select D# From Dept Where Dname = '计算机');

谢谢大家!

